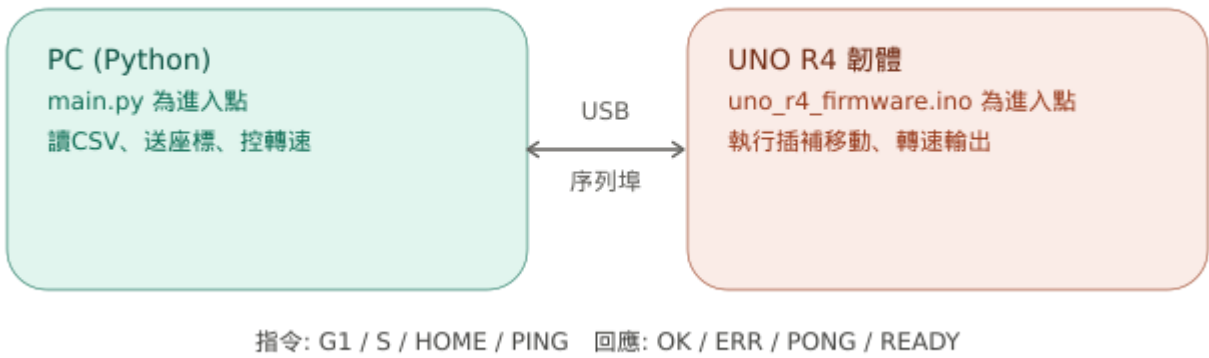


半自動光學鏡片拋光機 — 控制系統架構

本文件說明 PC(Python) 與 UNO R4 韌體之間的分工方式、模組相依關係與通訊協定，對應的原始碼位於 `pc/` 與 `uno_r4_firmware/` 資料夾。

1. 系統總覽

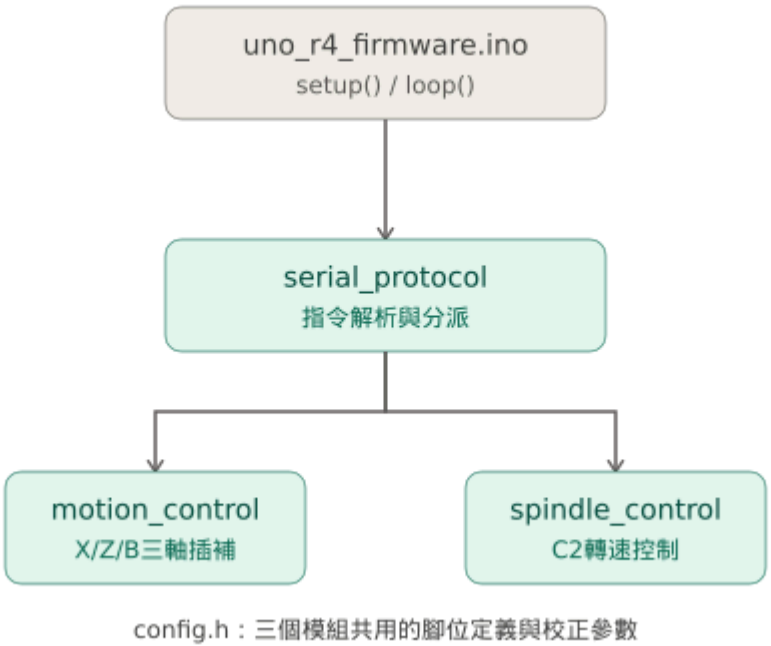


- **PC(Python)**：負責視覺辨識、CSV非球面座標讀取、製程邏輯判斷、人機介面、資料記錄
- **UNO R4韌體**：負責接收指令、直接驅動馬達(X/Z/B三軸)與拋光頭自轉(C2)、回報執行狀態
- 兩者透過 **USB序列埠(115200 bps)** 溝通，這是唯一的耦合點——只要通訊協定不變，兩邊的內部實作可以各自獨立替換(例如PC端換語言、UNO R4換成別的控制板)

通訊協定

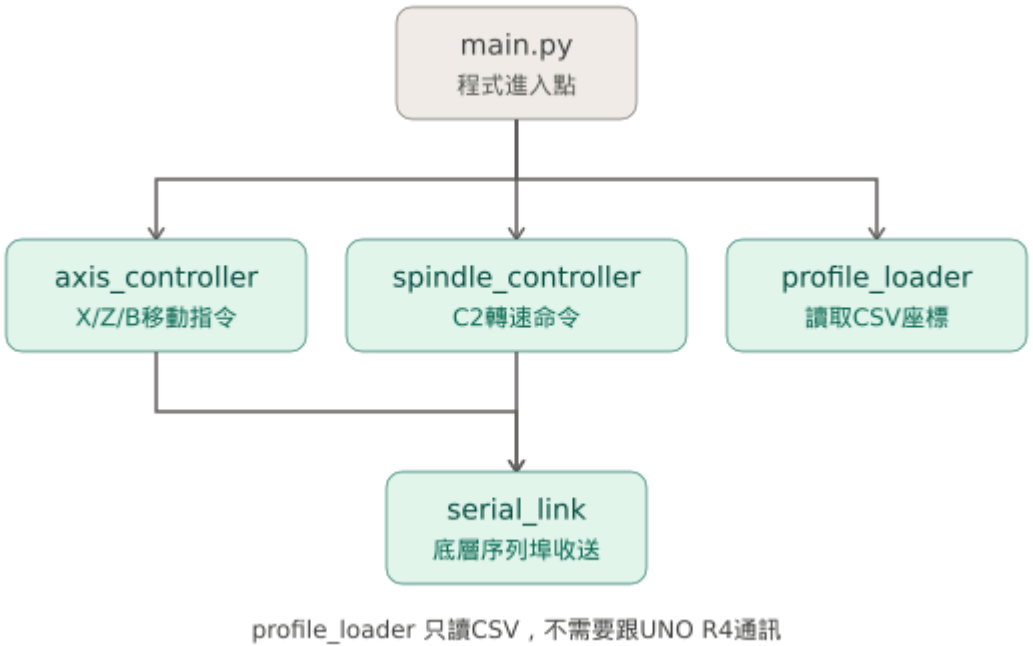
方向	指令/回應	說明
PC → UNO	G1 X<mm> Z<mm> B<deg>	移動到絕對座標(物理單位，換算成pulse在韌體端進行)
PC → UNO	S <0~100>	拋光頭自轉(C2)轉速命令
PC → UNO	HOME	三軸回原點
PC → UNO	PING	心跳/存活檢查
UNO → PC	OK / ERR <msg>	動作完成 / 指令錯誤
UNO → PC	PONG	回應PING
UNO → PC	READY	開機完成

2. UNO R4 韌體模組結構



檔案	職責
uno_r4_firmware.ino	主檔，只有 <code>setup()</code> / <code>loop()</code> ，呼叫其他模組的初始化與輪詢函式
serial_protocol.h/.cpp	解析PC送來的文字指令，分派給對應模組執行；含通訊逾時失效保護
motion_control.h/.cpp	X/Z/B三軸同步插補移動(AccelStepper + MultiStepper)、回原點
spindle_control.h/.cpp	C2拋光頭自轉轉速控制(目前為PWM佔位實作，待BLDC/VFD選型後改寫)
config.h	三個模組共用的腳位定義、校正參數(STEPS_PER_MM/DEG)、運動參數

3. PC(Python) 模組結構



檔案	職責
<code>main.py</code>	程式進入點，組裝以下模組，執行完整拋光流程
<code>serial_link.py</code>	底層序列埠收送，所有跟UNO R4的通訊都經由這個類別
<code>axis_controller.py</code>	X/Z/B高階操作介面(<code>move_to</code> 、 <code>home</code>)，內部呼叫 <code>serial_link</code>
<code>spindle_controller.py</code>	C2轉速命令介面(<code>set_speed</code> 、 <code>stop</code>)，內部呼叫 <code>serial_link</code>
<code>profile_loader.py</code>	讀取非球面CSV座標檔，不需要跟UNO R4通訊
<code>config.py</code>	設定值(序列埠埠號、CSV路徑等)

4. 完整檔案結構

```

polishing_v2/
├── docs/
│   ├── ARCHITECTURE.md      本文件
│   └── diagrams/            架構圖SVG
├── uno_r4_firmware/
│   ├── uno_r4_firmware.ino
│   ├── config.h
│   ├── motion_control.h/.cpp
│   ├── spindle_control.h/.cpp
│   └── serial_protocol.h/.cpp
└── pc/
    ├── config.py
    ├── serial_link.py
    ├── axis_controller.py
    ├── spindle_controller.py
    ├── profile_loader.py
    ├── main.py
    └── lens_profile_example.csv

```

5. 已知待辦事項

- `spindle_control.cpp` 目前為PWM佔位邏輯，待C2軸最終選型(BLDC控制器 / VFD+AC馬達)確定後改寫，對外呼叫介面(`spindleSetSpeed`、`spindleStop`)不需要更動
- `STEPS_PER_MM_X`、`STEPS_PER_MM_Z`、`STEPS_PER_DEG_B` 為佔位校正值，務必依實際機構(驅動器脈波數/減速比/導程)實測校正後才能通電測試
- 通訊逾時失效保護(`serial_protocol.cpp` 內的 `TODO`)目前只停止主軸，建議依製程需求補上更完整的安全動作(例如Z軸自動抬升脫離鏡片)
- 未來可擴充：視覺辨識(拋光布磨損檢測)建議獨立成 `vision_inspector.py`，在 `main.py` 中匯入呼叫，維持模組職責單一的架構原則